12. Deep Learning 5

Advanced Methods

Machine learning

Jiri Chmelik
Department of Biomedical Engineering

Introduction

- Generative Adversarial Network (GAN)
  - Basic principle
  - Wasserstein GAN (WGAN)
  - Conditional GAN (cGAN)
- Graph Convolutional Networks (GCN)
- Siamese Networks
- Transformers

Introduction

- Generative Adversarial Network (GAN)
  - Basic principle
  - Wasserstein GAN (WGAN)
  - Conditional GAN (cGAN)
- Graph Convolutional Networks (GCN)
- Siamese Networks
- Transformers

Generative Adversarial Network

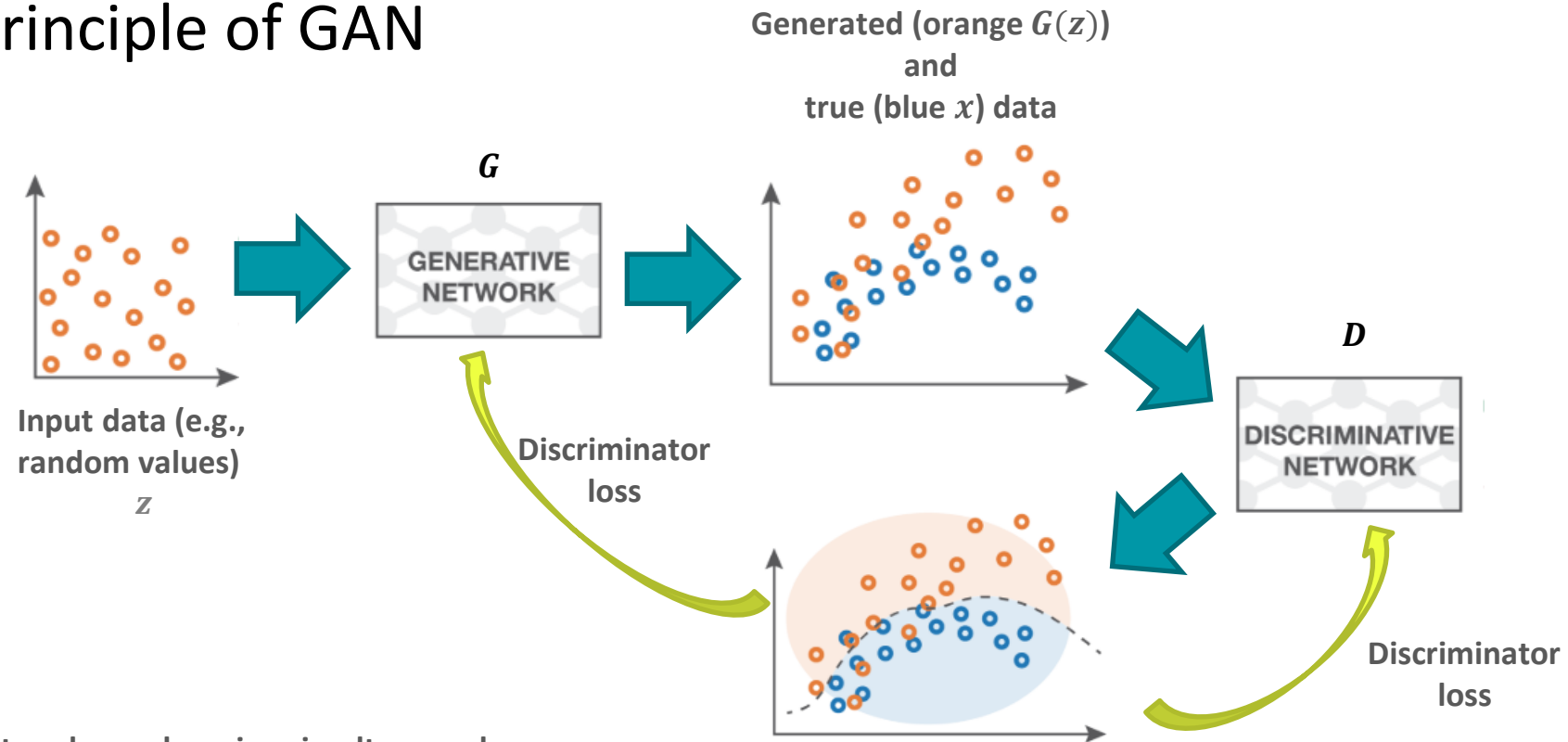
- The main idea is to combine two networks, which are aiming to opposite goals and they thus compete with each other.

- Two neural networks:
  – generator (decoder, recurrent network, image-to-image, …)
  – discriminator (classifier)

- The first (generative) networks is trained to **maximize** error of discriminator, and on the contrary, the discriminator is trained to **minimize** discriminator error.

- There are many modification (e.g., conditional GANs, Wasserstein GAN, CycleGAN, …).

Principle of GAN



Networks are learning simultaneously, where the class. error is used for both nets

Principle of GAN

GAN optimization – Discriminator by binary cross-entropy (1 – real, 0 – fake)

$$\max_D V(D) = \underbrace{E_{\boldsymbol{x}\sim p_{data}(\boldsymbol{x})}[\log(D(\boldsymbol{x}))]}_{\text{Real inputs}} + \underbrace{E_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}\left[\log\left(1-D\left(G(\boldsymbol{z})\right)\right)\right]}_{\text{Fake (generated) inputs}}$$

GAN optimization – Generator – to “fool” the discriminator to produce high value on fake input

$$\min_G V(G) = E_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}\left[\log\left(1-D\left(G(\boldsymbol{z})\right)\right)\right]$$

GAN optimization – Combined – Competition between generator and discriminator

$$\min_G \max_D V(D,G) = E_{\boldsymbol{x}\sim p_{data}(\boldsymbol{x})}[\log(D(\boldsymbol{x}))] + E_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}\left[\log\left(1-D\left(G(\boldsymbol{z})\right)\right)\right]$$

GAN training

GAN algorithm – Discriminator update by mini-batch SGD

$$\theta_D^{it} = \theta_D^{it-1} + \mu_D \nabla_{\theta_D} \frac{1}{m} \sum_{i=1}^{m} \log\left(D\left(\boldsymbol{x}^i\right)\right) + \log\left(1 - D\left(G\left(\boldsymbol{z}^i\right)\right)\right)$$

Gradient of **discriminator** loss

Going in **ascending** direction – maximization

GAN algorithm – Generator update by mini-batch SGD

$$\theta_G^{it} = \theta_G^{it-1} - \mu_G \nabla_{\theta_G} \frac{1}{m} \sum_{i=1}^{m} \log\left(1 - D\left(G\left(\boldsymbol{z}^i\right)\right)\right)$$

Gradient of **generator** loss

Going in **descending** direction – minimization

https://poloclub.github.io/ganlab/

GAN training

- Training is repeated until stopping criteria
- Discriminator is usually trained more times than generator in each iteration – discriminator update is repeated k-times (hyperparameter) while generator is updated only once
- Discriminator and generator usually have different architectures and hyperparameters
- Other learning algorithms could be used for training

GAN algorithm – modification of generator update – diminished gradient problem – no training if discriminator wins against generator, especially in early training.

$$\theta_G^{it} = \theta_G^{it-1} + \mu_G \nabla_{\theta_G} \frac{1}{m} \sum_{i=1}^{m} \log\left(D\left(G\left(\boldsymbol{z}^i\right)\right)\right)$$

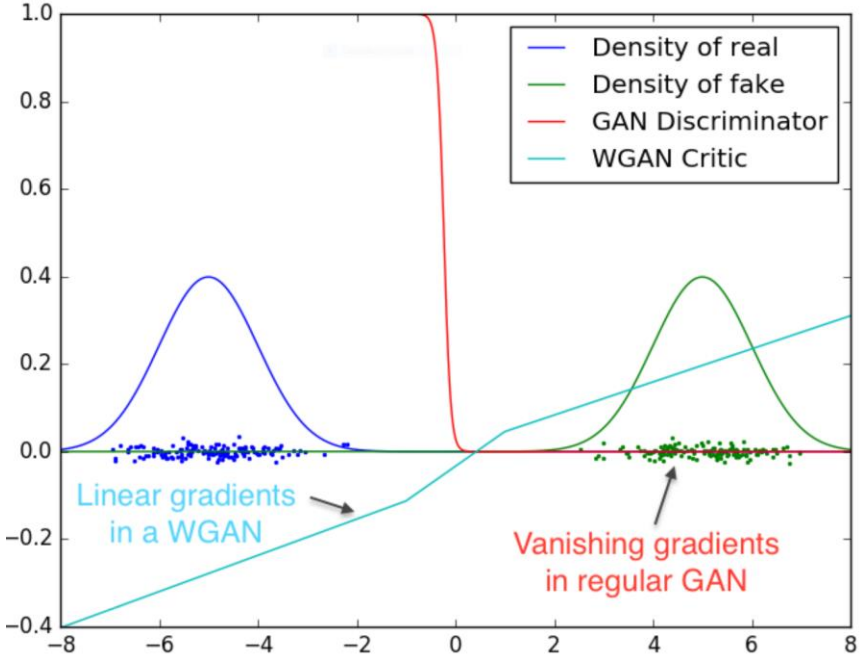
However, the problem with stability in cases of high variance of gradients – solution: adding a noise to generated images or new cost function to smooth the gradient – **Wasserstein Distance.**

Introduction

- Generative Adversarial Network (GAN)
  - Basic principle
  - Wasserstein GAN (WGAN)
  - Conditional GAN (cGAN)
- Graph Convolutional Networks (GCN)
- Siamese Networks
- Transformers

Why Wasserstein distance?

- Problem of diminished gradient due to good quality of discriminator



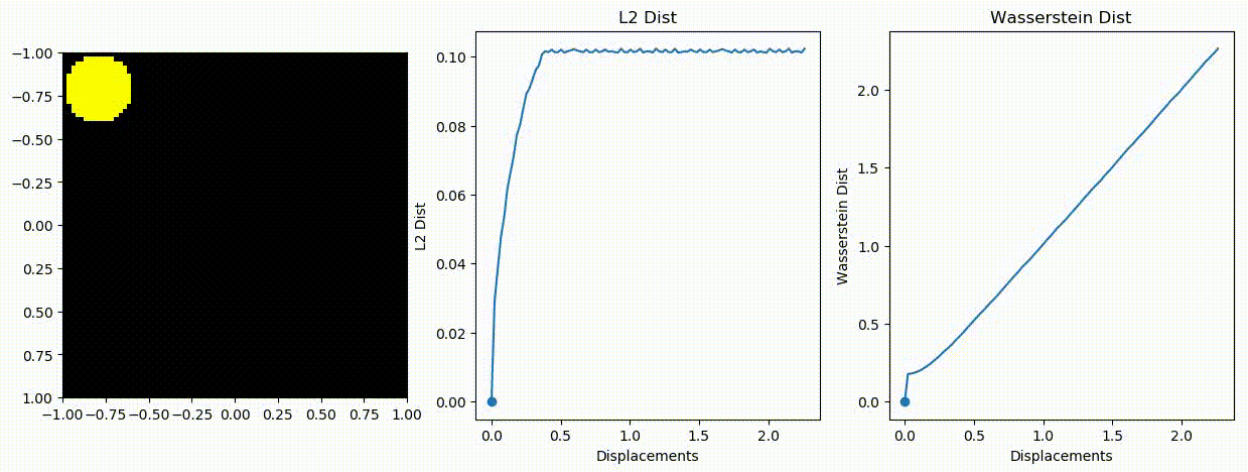
Source: ARJOVSKY, Martin; CHINTALA, Soumith; BOTTOU, Léon. Wasserstein gan. *arXiv preprint arXiv:1701.07875*, 2017.

Wasserstein distance

Wasserstein distance (Earth-Mover Distance)

- Minimum cost of transporting mass converting the mass distributions
- In our case converting the real data distribution P_r to the generated data distribution P_g

$$W(P_r, P_g) = \inf_{\gamma \in \Gamma(P_r, P_g)} E_{(x,y)\sim\gamma}[\|x - y\|]$$



source: https://gist.github.com/ctralie/66352ae6ab06c009f02c705385a446f3

Lipschitz condition

Lipschitz continuity condition

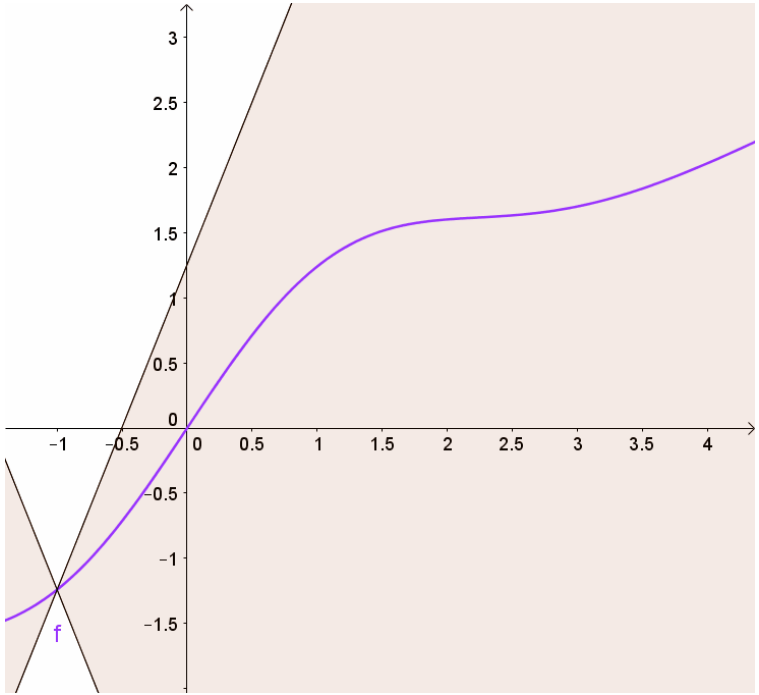
- is valid for a real function f if:

$$|f(x_1) - f(x_2)| \leq K|x_1 - x_2| \qquad \forall\, x_1, x_2 \in R$$

- Absolute value of slope of function $|f(x_1) - f(x_2)|$ is always smaller than K –> if a function has bounded first derivates, it is Lipschitz

$$\frac{|f(x_1) - f(x_2)|}{|x_1 - x_2|} \leq K$$

Lipschitz constant



source: https://commons.wikimedia.org/wiki/File:Lipschitz_Visualisierung.gif

Principle of WGAN

Wasserstein distance in WGAN application

- WD can be simplified using Kantorovich-Rubinstein duality to:

sup is the least upper bound

$$W(P_r, P_g) = \sup_{\|f\|_L \leq 1} E_{x \sim P_r}[f(x)] - E_{x \sim P_g}[f(x)]$$

Lipschitz constant K=1

- Thus, we want to have value of f high if $P_r \geq P_g$ and low if $P_r < P_g$
- Now, we can use WD as a loss function instead of cross-entropy loss

WGAN training

WGAN algorithm – Critic (discriminator in GAN) update by mini-batch SGD

$$\theta_C^{it} = \theta_C^{it-1} + \mu_C \nabla_{\theta_C} \frac{1}{m} \sum_{i=1}^{m} f(\boldsymbol{x}^i) - f\left(G(\boldsymbol{z}^i)\right)$$

Gradient of **critic** loss

Going in **ascending** direction – maximization

GAN algorithm – Generator update by mini-batch SGD

$$\theta_G^{it} = \theta_G^{it-1} - \mu_G \nabla_{\theta_G} \frac{1}{m} \sum_{i=1}^{m} f\left(G(\boldsymbol{z}^i)\right)$$

Gradient of **generator** loss

Going in **descending** direction – minimization

WGAN training

What is the problem?

- How to do the function f to be the 1-Lipschtz?
- Just **clip** the parameters of the critic with hyperparameter c (values of all θ_C^{it} will be equal to $-c$ if lower than $-c$ and equal to c if higher than c:

$$\theta_C^{it} = clip\left(\theta_C^{it}, -c, c\right)$$

It means that the maximal changes of the critic parameters are controlled by clipping hyperparameter (the change will never be higher than c).

WGAN Summary

Notes for clipping the weights:

- **Very sensitive to clipping hyperparameter:**
- **If too large clipping hyperparameter, it takes long to reach the limit – no optimal training of critic or even exploding gradient.**
- **If too small – vanishing gradients or too slow training.**
- **It behaves as a model regularization – lower c means lower capacity of model.**
- **Very simple and easy to implement.**

Solution – WGAN-GP (Gradient Penalty) modification

- **Norm of the gradient should be equal to 1 to achieve 1-Lipschitz.**
- **Achieved by random uniform mixing of real and fake data points (data are interpolated between somewhere between real and fake data point).**
- **And instead of clipping the model is penalized by gradient L2 norm different from 1.**

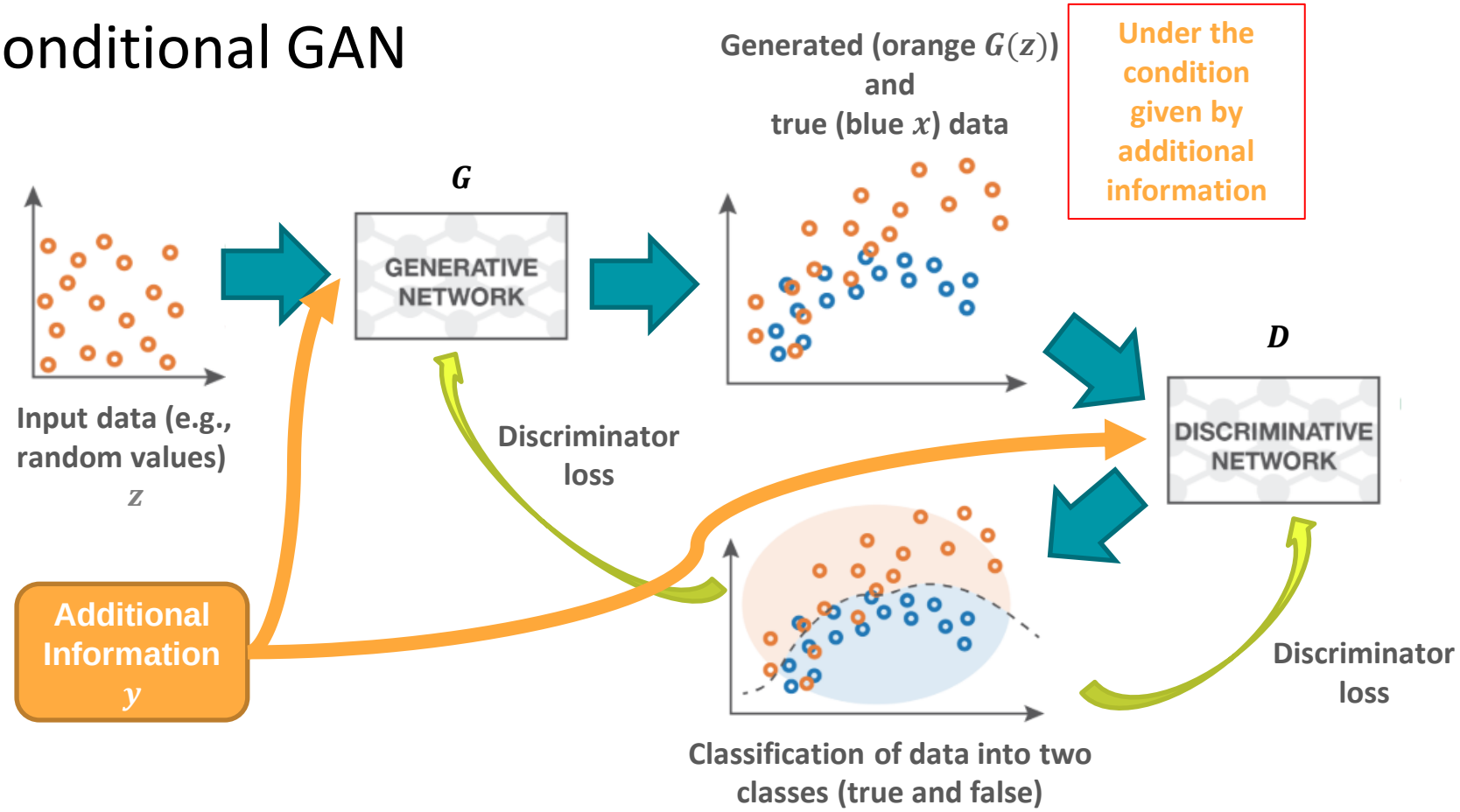
$$\hat{\boldsymbol{x}} = \epsilon \boldsymbol{x} + (1-\epsilon) G(\boldsymbol{z})$$

$$L = D(G(\boldsymbol{z})) - D(\boldsymbol{x}) + \lambda(\|\nabla_{\hat{\boldsymbol{x}}} D(\hat{\boldsymbol{x}})\|_2 - 1)^2$$

Introduction

- Generative Adversarial Network (GAN)
  - Basic principle
  - Wasserstein GAN (WGAN)
  - Conditional GAN (cGAN)
- Graph Convolutional Networks (GCN)
- Siamese Networks
- Transformers

Conditional GAN



Principle of cGAN

GAN optimization

$$\min_G \max_D V(D,G) = E_{\boldsymbol{x}\sim p_{data}(\boldsymbol{x})}[\log(D(\boldsymbol{x}))] + E_{\boldsymbol{z}\sim p_{\boldsymbol{z}}(\boldsymbol{z})}\left[\log\left(1 - D\left(G(\boldsymbol{z})\right)\right)\right]$$

cGAN optimization

$$\min_G \max_D V(D,G) = E_{\boldsymbol{x},\boldsymbol{y}\sim p_{data}(\boldsymbol{x},\boldsymbol{y})}[\log(D(\boldsymbol{x},\boldsymbol{y}))] + E_{\boldsymbol{z},\boldsymbol{y}\sim p_{\boldsymbol{z}}(\boldsymbol{z},\boldsymbol{y})}\left[\log\left(1 - D(G(\boldsymbol{z},\boldsymbol{y}),\boldsymbol{y})\right)\right]$$

$\boldsymbol{y}$ – additional information

cGAN optimization with similarity preservation (L1 norm, but L2 norm is also possible)

$$\min_G \max_D V(D,G) = E_{\boldsymbol{x},\boldsymbol{y}}[\log(D(\boldsymbol{x},\boldsymbol{y}))] + E_{\boldsymbol{z},\boldsymbol{y}}\left[\log\left(1 - D(G(\boldsymbol{z},\boldsymbol{y}),\boldsymbol{y})\right)\right] + \lambda E_{\boldsymbol{x},\boldsymbol{y},\boldsymbol{z}}[\|\boldsymbol{x} - G(\boldsymbol{z},\boldsymbol{y})\|_1]$$

λ – additional weighting hyperparameter

cGAN Summary

Properties:

- **Can be used as specific generator (specific class, specific image, etc.).**
- **Comparing to standard GAN, cGAN preserves important information (style, positions, class, etc.).**
- **Can use advantages of WGAN training.**
- **High level of abstraction – risky to use if you need to know how it works or what is truly generated.**
- **Inappropriate to generation of data with need of high sense of reality (medical images, aviation, driving, military use, etc.). –> nearly unusable for exact and objective applications.**

Another modifications:

- **InfoGAN (Information Maximizing) modification – discriminator has not binary classification only, but also, for example, predicts classes. Moreover, it enables unsupervised learning based on mutual information loss.**
- **CycleGAN modification – unpaired Image-to-Image translation. Combination of 2 GAN models with reversal functions trained together in forward/backward manner.**

Utilizing of GANs

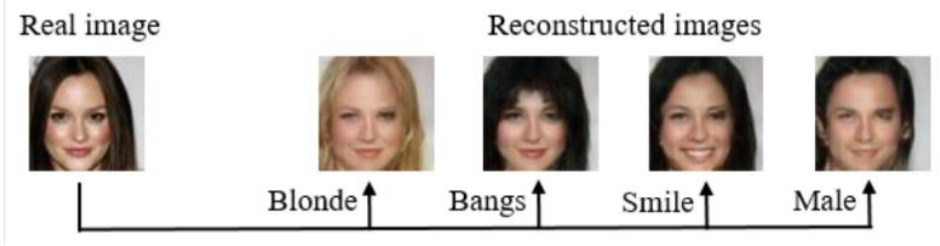
- **Generate Examples for Image Datasets**
- **Generate Photographs of Human Faces**
- **Generate Realistic Photographs**
- **Generate Cartoon Characters**
- **Image-to-Image Translation**
- **Text-to-Image Translation**
- **Semantic-Image-to-Photo Translation**
- **Face Frontal View Generation**
- **Generate New Human Poses**
- **Photos to Emojis**
- **Photograph Editing**
- **Face Aging**
- **Photo Blending**
- **Super Resolution**
- **Photo Inpainting**
- **Clothing Translation**
- **Video Prediction**
- **3D Object Generation**
- **…**

http://gandissect.res.ibm.com/ganpaint.html?project=churchoutdoor&layer=layer4

https://machinelearningmastery.com/impressive-applications-of-generative-adversarial-networks/

https://jonathan-hui.medium.com/gan-some-cool-applications-of-gans-4c9ecca35900

GAN Examples



Introduction

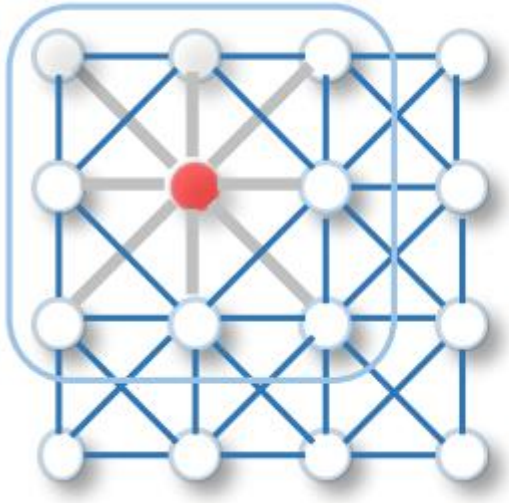
- Generative Adversarial Network (GAN)
  - Basic principle
  - Wasserstein GAN (WGAN)
  - Conditional GAN (cGAN)
- Graph Convolutional Networks (GCN)
- Siamese Networks
- Transformers

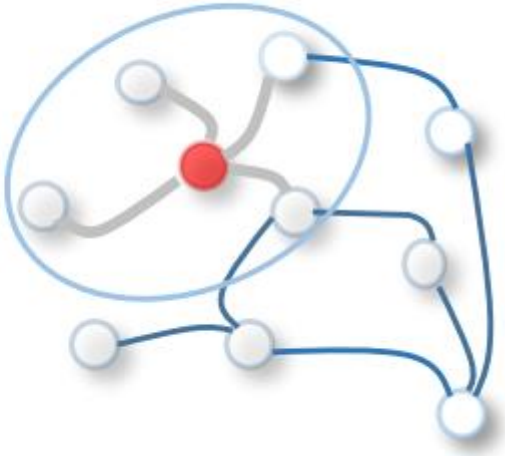
Graph Convolutional Network

- Convolution neighbourhood is dependent on graph instead of regular net.

$G \triangleq (V, \mathrm{E})$

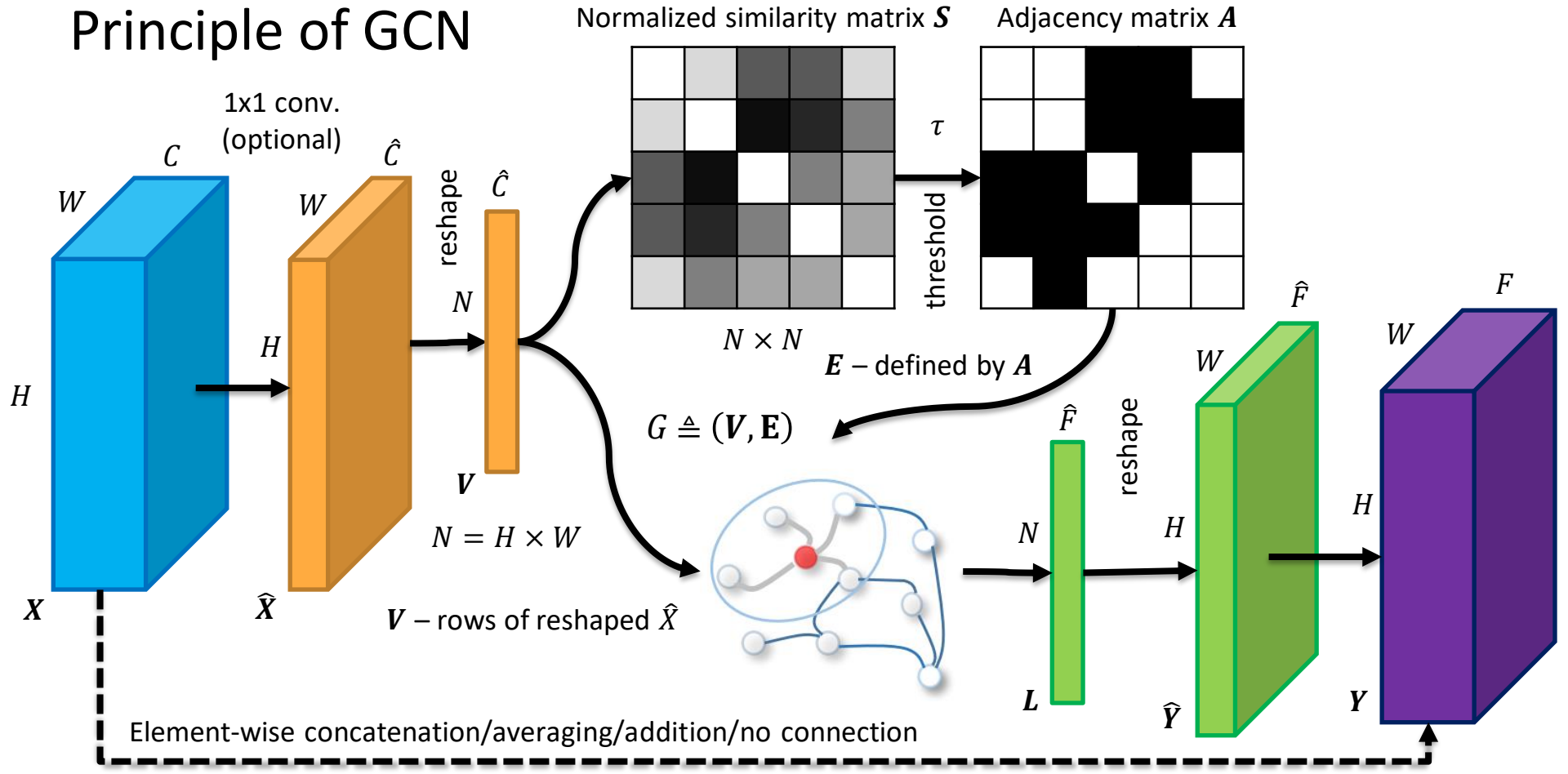
G – graph
P – vertices (nodes)
E – edges (connections)





Source: WU, Zonghan, et al. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 2020.

Principle of GCN



Principle of GCN

- Computation of GCN output $\boldsymbol{L}$ via spectral (Laplacian) convolution in matrix form (final term only)

$$\mathbf{L} = f(\boldsymbol{D}^{-\frac{1}{2}}\boldsymbol{A}\boldsymbol{D}^{-\frac{1}{2}}\boldsymbol{V}\boldsymbol{W})$$

$$\boldsymbol{D}_{i,i} = \sum_{j} \boldsymbol{A}_{i,j}$$

$\boldsymbol{D} \in R^{N \times N}$ – diagonal matrix of node (ventricle) degrees
$\boldsymbol{A} \in R^{N \times N}$ – symmetric adjacent matrix with ones on the main diagonal (self-connections)
$\boldsymbol{V} \in R^{N \times \hat{C}}$ – reshaped input matrix
$\boldsymbol{W} \in R^{\hat{C} \times \hat{\boldsymbol{F}}}$ – trainable weight matrix
$\boldsymbol{L} \in R^{N \times \hat{\boldsymbol{F}}}$ – trainable weight matrix (Laplacian matrix)
f – activation function

Details in: KIPF, Thomas N.; WELLING, Max. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.

GCN Summary

Properties:

- **Generalized form of CNNs.**
- **Can reveal/use relations between input points or hidden features.**
- **Usability in problems, where the relations between nodes are important (chemistry, dynamic systems, computer vision of living subjects, etc.).**
- **Higher computational complexity.**
- **Higher level of abstraction.**

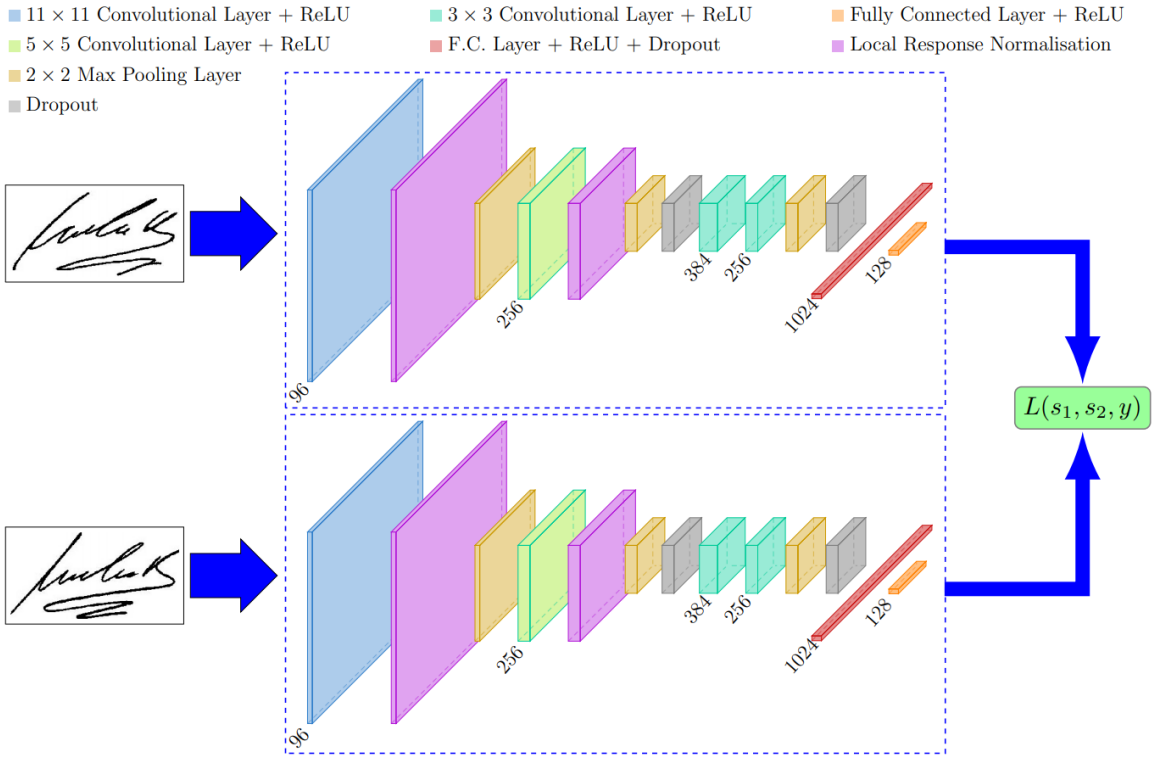
Another modifications and upgrades:

- **RecGCN, GAE, spatio-temporal, pyramidal GCN, spectral GCN, spatial GCN, diffusion GCN, etc.**
- **Graph pooling, …**

Introduction

- Generative Adversarial Network (GAN)
  - Basic principle
  - Wasserstein GAN (WGAN)
  - Conditional GAN (cGAN)
- Graph Convolutional Networks (GCN)
- Siamese Networks
- Transformers

Siamese Neural Networks



Source: DEY, Sounak, et al. Signet: Convolutional siamese network for writer independent offline signature verification. *arXiv preprint arXiv:1707.02131*, 2017.

Siamese Neural Networks

Triplet loss

$$L(A, P, N) = max(\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + m, 0)$$

A – anchor point
P – positive point
N – negative point
f – function of NN
m – marginal term

Contrastive loss

$$L(\theta, (y, \boldsymbol{x}_1, \boldsymbol{x}_2)) = (1 - y)\frac{1}{2}(D_\theta)^2 + y\frac{1}{2}\{max(0, m - D_\theta)\}^2$$

$$D_\theta = \sqrt{\{G_\theta(\boldsymbol{x}_1) - G_\theta(\boldsymbol{x}_2)\}^2}$$

θ – NN parameters
y – is dissimilar?
$\boldsymbol{x}$ – input data
D – Euclidean distance
m – marginal term
G – output of NN

- Two forward passes only
- Nearest neighbour for new image – is lower than m?

Siamese Neural Networks Summary

Properties:

- **Used for comparison of inputs.**
- **No need of heuristic feature selection.**
- **Applications in security, identification, recognition, etc.**
- **After modifications used in segmentation, classification, and other standard applications.**
- **High prediction times based on number of used copies (during training in forward feeding only).**

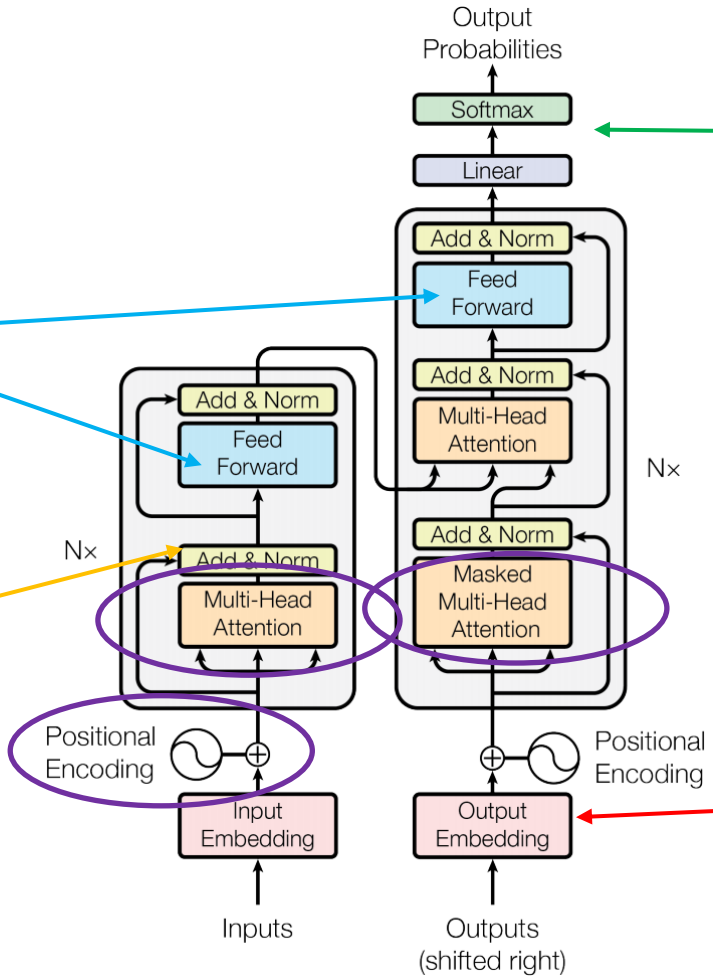
Introduction

- Generative Adversarial Network (GAN)
  - Basic principle
  - Wasserstein GAN (WGAN)
  - Conditional GAN (cGAN)
- Graph Convolutional Networks (GCN)
- Siamese Networks
- Transformers

Transformers

Standard
2-layer feedforward
fully connected
network with ReLU
and linear activations

Residual connections
with layer normalization

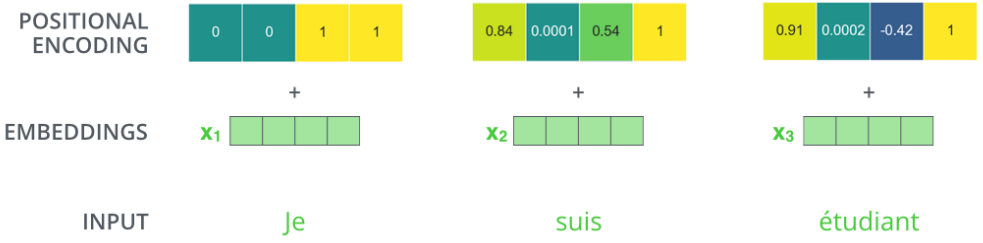


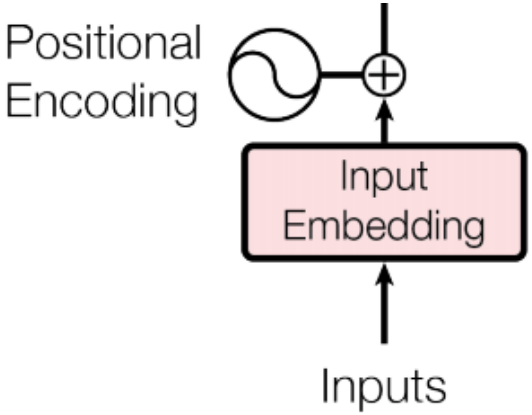
Classifier – output probability vector with length given by number of words in vocabulary

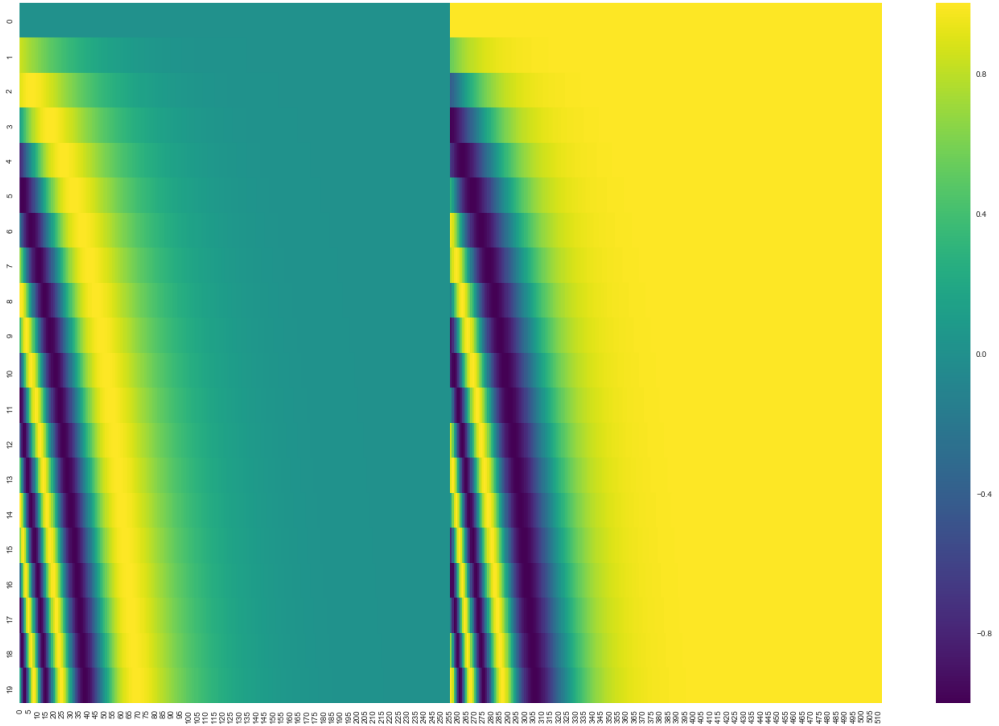
Word2Vec, Skip-Gram, …
Convert the word to numeric vector
Simple 2-leayer net
https://ronxin.github.io/wevi/

Source: VASWANI, A., et al. Attention is all you need. arXiv 2017. *arXiv preprint arXiv:1706.03762*, 2017.

Transformers

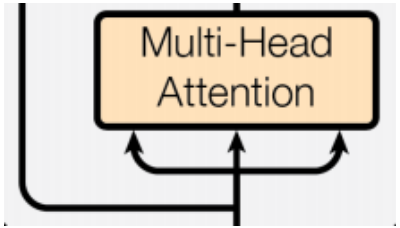






Source: http://jalammar.github.io/illustrated-transformer/

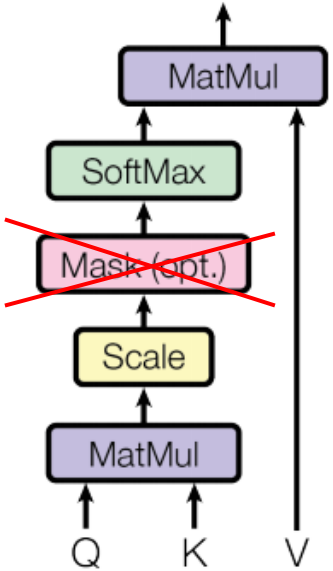
Transformers



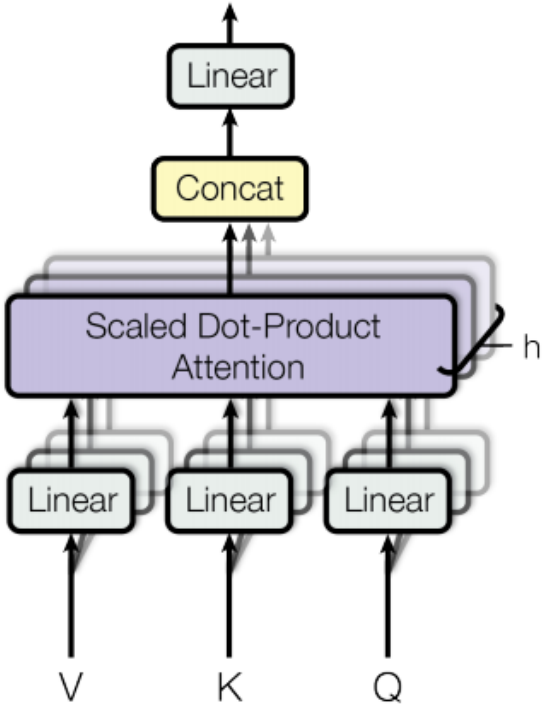
$$MHA(Q,K,V) = Concat(head_1, head_2, \dots head_h)W^O$$
$$head_i = A(QW_i^Q, KW_i^K, VW_i^V)$$

Scaled Dot-Product Attention

$$A(Q,K,V) = SoftMax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



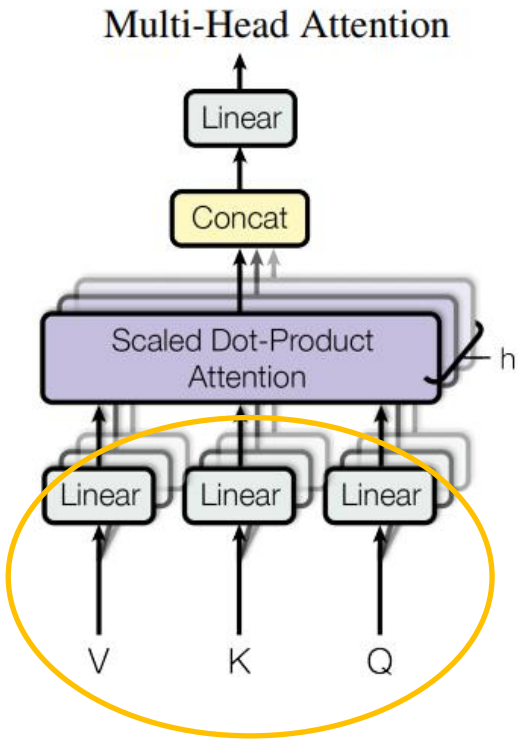
Multi-Head Attention

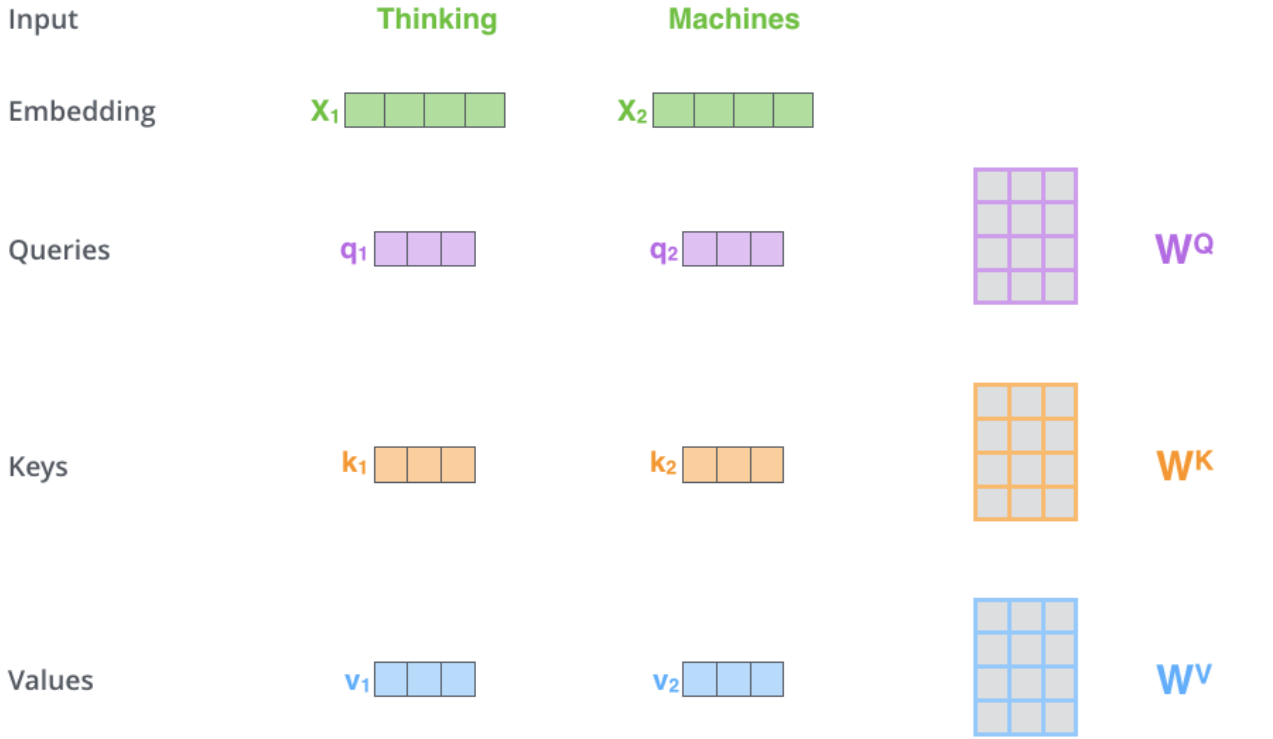


Q – query
K – key
V – value
d_k – dimension of key
MatMul – Marix implementation of Dot Product

Source: VASWANI, A., et al. Attention is all you need. arXiv 2017. *arXiv preprint arXiv:1706.03762*, 2017.

Transformers

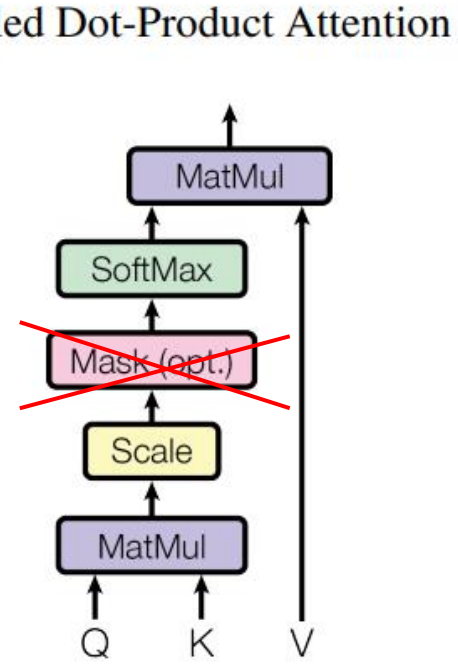


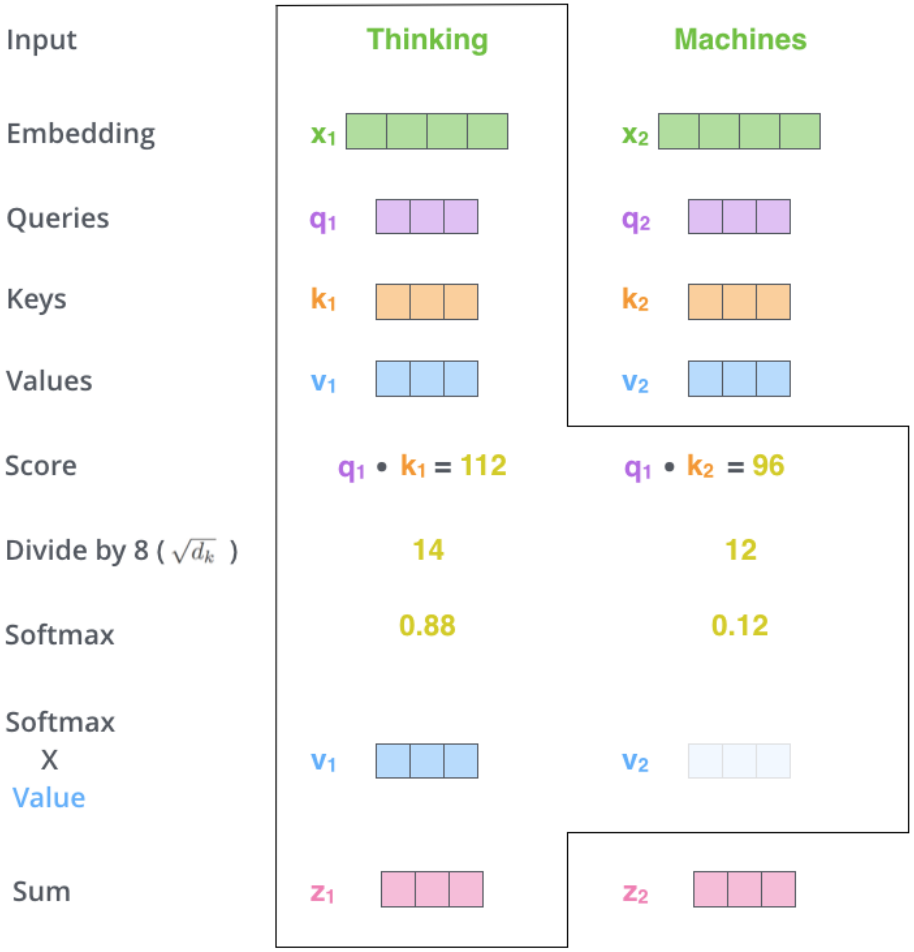


Multiplying x1 by the WQ weight matrix produces q1, the "query" vector associated with that word. We end up creating a "query", a "key", and a "value" projection of each word in the input sentence.

Source: http://jalammar.github.io/illustrated-transformer/

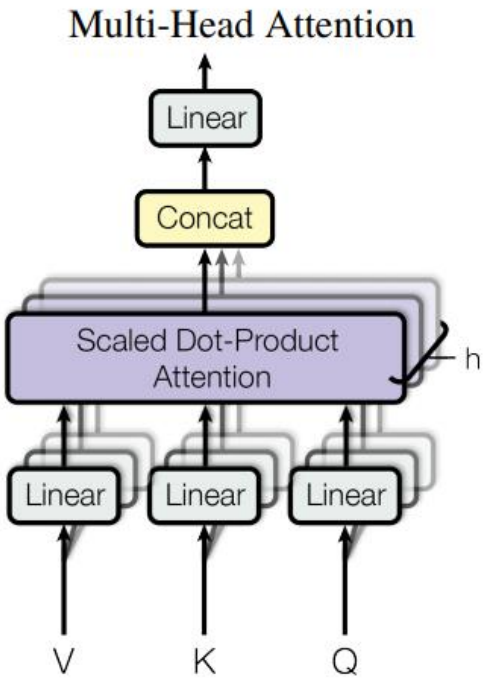
Transformers





Source: http://jalammar.github.io/illustrated-transformer/

Transformers



1) This is our input sentence*

2) We embed each word*

Thinking Machines

X

* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one

R

3) Split into 8 heads. We multiply X or R with weight matrices

W_0^Q W_0^K W_0^V

W_1^Q W_1^K W_1^V

...

W_7^Q W_7^K W_7^V

4) Calculate attention using the resulting Q/K/V matrices

Q_0 K_0 V_0

Q_1 K_1 V_1

...

Q_7 K_7 V_7

5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

Z_0

Z_1

...

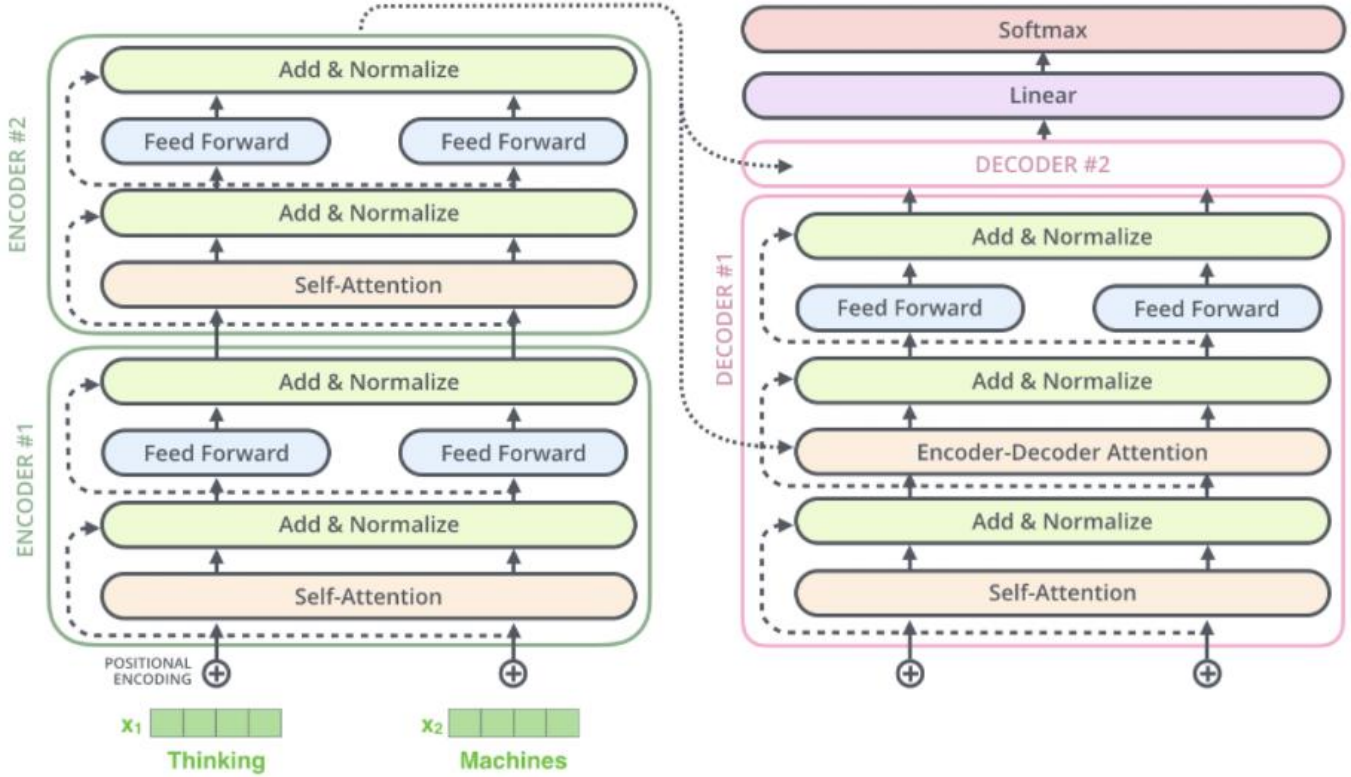
Z_7

W^O

Z

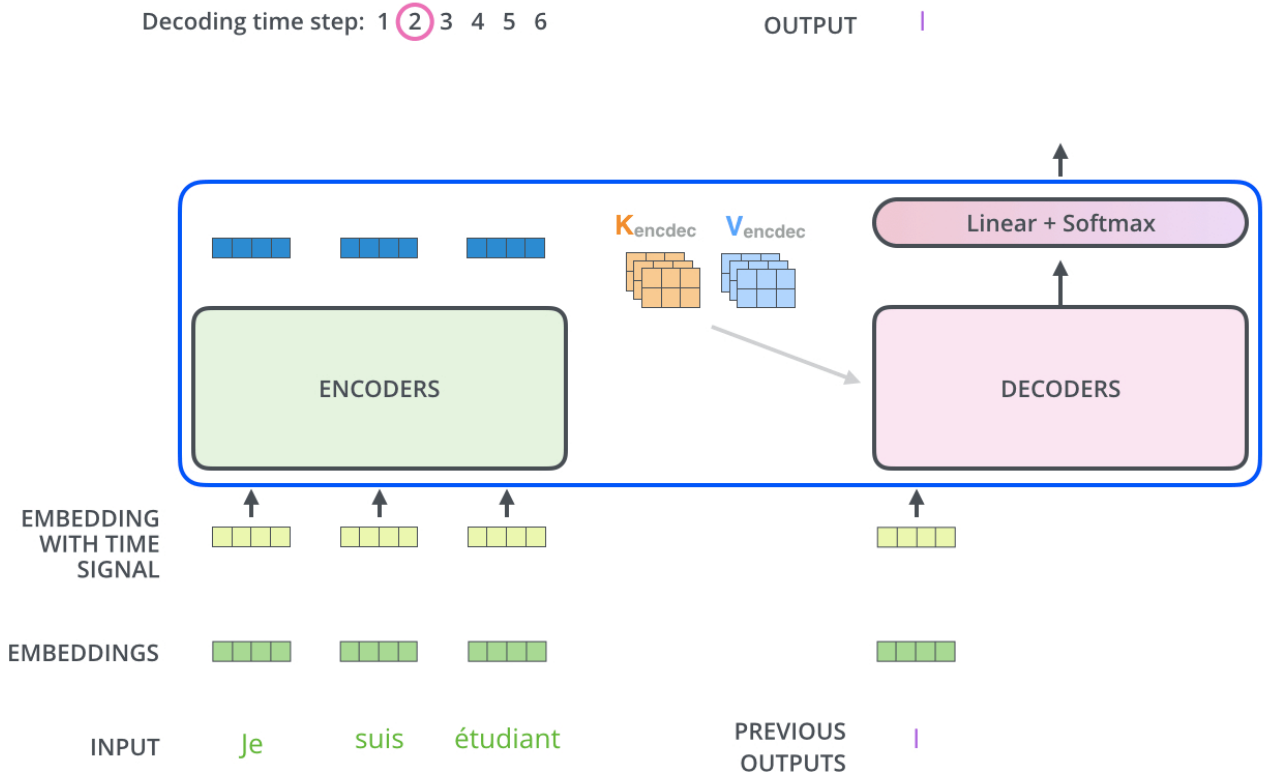
Source: http://jalammar.github.io/illustrated-transformer/

Transformers



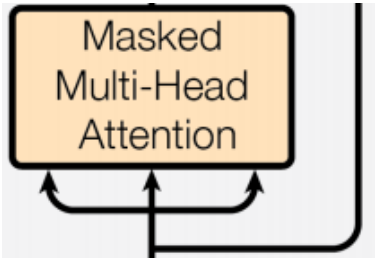
Source: http://jalammar.github.io/illustrated-transformer/

Transformers



Source: http://jalammar.github.io/illustrated-transformer/

Transformers



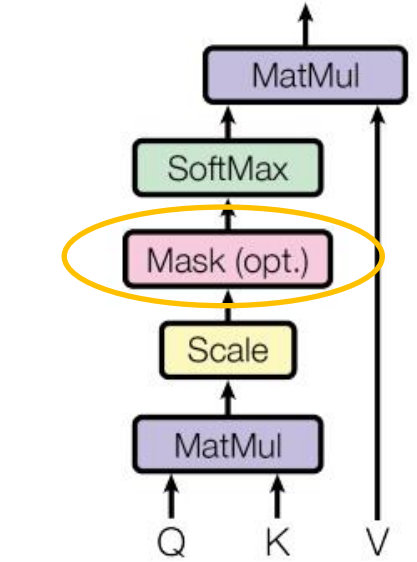
14	10	5
12	28	17
8	32	28

+

0	-∞	-∞
0	0	-∞
0	0	0

=

14	-∞	-∞
12	28	-∞
8	32	28

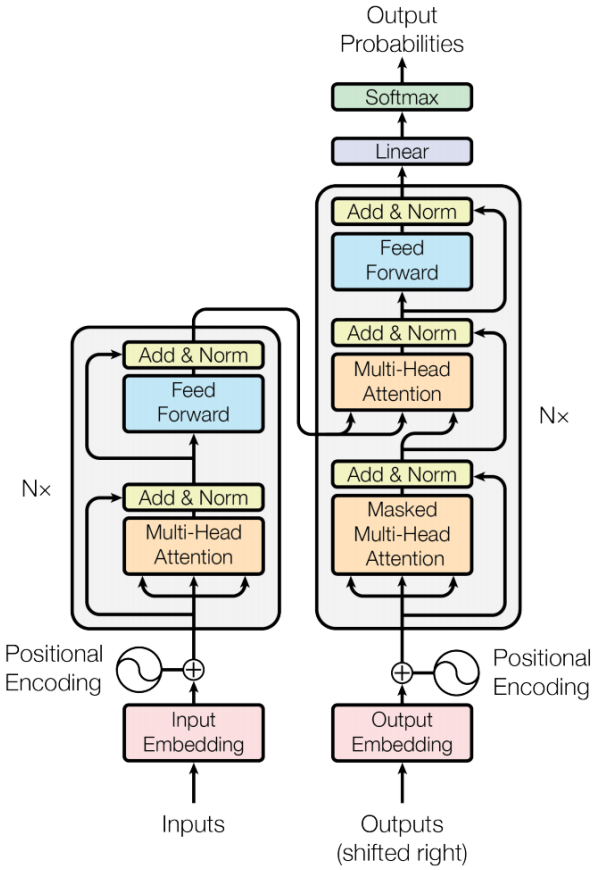


Source: http://jalammar.github.io/illustrated-transformer/

Transformers

Training

- Input is a whole sentence (e.g., in one language).
- Output is the word in the target (different) language, which follows the processed input word.
- It means the decoder is traying to predict the next word knowing whole sentence in one language and the previous (already predicted) words in other language.
- It trains the context of words not only their meaning.
- Standard backpropagation in iterations through whole vocabulary.
- Also, different training options exist.



Source: VASWANI, A., et al. Attention is all you need. arXiv 2017. *arXiv preprint arXiv:1706.03762*, 2017.

Transformers Summary

Properties:

- **Huge computational complexity (basic model, 3.5 days on 8 Tesla P100)**
- **Need of vocabulary.**
- **Can’t process very long or infinite signals.**
- **Can’t run in real time.**
- **Encoder and decoder have access to all quarry, keys and values and attention just take the important ones – replace of memory in recurrent networks.**
- **Can be pre-trained and fine-tunned!**
- **No recurrent block – parallelization is possible!**

Pre-trained transformer models:

- **NVIDIA Megatron-LM**
- **OpenAI GPT, GPT-2, GPT-3 (175 billions of params., 355 years on Tesla V100, 4.6M$)**
- **Google BERT**
- **Facebook RoBERTa**
- **Microsoft Turing-NLG**
- **Transformer-XL, XLNet**